5 Strings

5.1 Introduction

A string is a text, i.e. a sequence of characters (letters, digits and other special characters). String handling is a little tricky in C++, so we have dedicated an separate chapter to this subject.

Actually, a string is an array that consists of a number of items, where each item is a character in the string.

There are a number of string handling functions. We will in this chapter get acquainted with the most common and usable string functions, like calculating the length of a string, copying a string, concatenating strings and picking out parts of a string.

In programming, string handling is important, since a user often enters information in text form which is taken care of by the program. We will go through several programming examples, where we will have use of our string knowledge.

5.2 Data Type char

We will first get to know the most simple of all strings, namely the one containing just one character. To store one character in a variable we use the data type char. Below we declare a string variable of the char type:

```
char cLetter;
```

The variable cLetter can now contain any one character. We can assign a value to the variable:

```
cLetter = 'A';
```

Note that we use single quotes for the character.

We can also, like for all kinds of variables, assign a value directly in the declaration:

```
char cLetter = 'A';
```

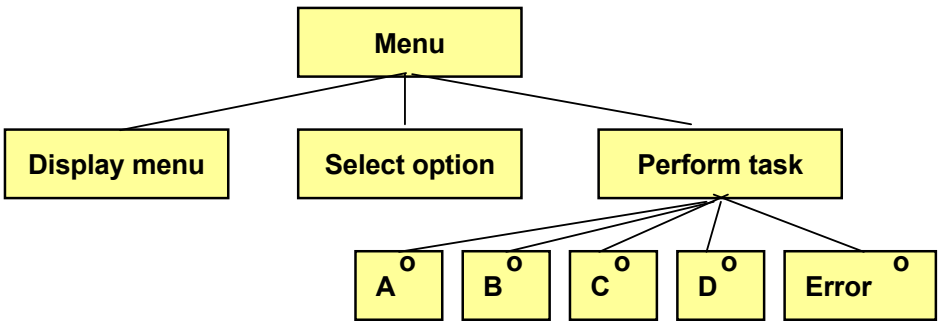
5.3 Menu Program

Our first program shows how to handle entry of characters by the user to select a menu option. The program will first display a menu:

```
   Menu
   ====
A. Order
B. Invoice
C. Warehouse
D. Finance
Select:
```

Here the user can enter one of the letters A, B, C or D to choose an item. We will not build a full-featured order/invoice/ warehouse/ finance system, but we will only let the program print a text about the selected choice. We start with a JSP graph:

Download free eBooks at bookboon.com



The first action is that the menu is displayed. Then the user is prompted for a choice by means of the letters A-D. Finally the requested option is executed, i.e. a simple text message will be printed. If the user enters another character than A-D, an error message is printed. Here is the code:

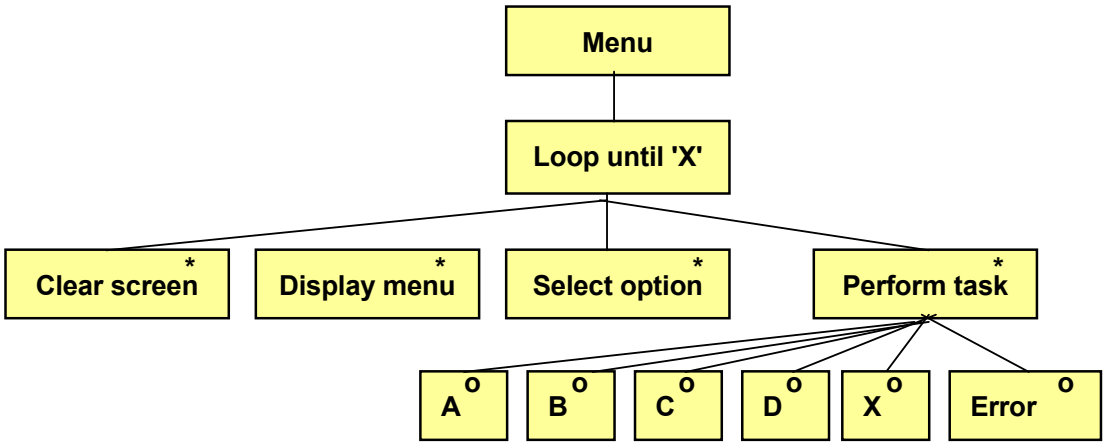
```
#include <iostream.h>
void main()
{
  char cSel;
      cout << "    Menu" << endl <<"    ===" << endl;
      cout << "A. Order" << endl;
      cout << "B. Invoice" << endl;
      cout << "C. Warehouse" << endl;
      cout << "D. Finance" << endl;
      cout << "Select: ";
      cin >> cSel;
      switch (cSel)
      {
    case 'A':
      cout << "You selected Order";
      break;
    case 'B':
      cout << "You selected Invoice";
      break;
    case 'C':
      cout << "You selected Warehouse";
      break;
    case 'D':
      cout << "You selected Finance";
      break;
    default:
      cout << "Erroneous choice";
      break;
  }
}
```


Download free eBooks at bookboon.com

First a char variable is declared named cSel, and then the menu is printed with a number of cout statements. The subsequent cin statement reads a character from the user to the variable cSel, which then is checked in the switch statement. The switch statement contains one case section for each option. Note that each case line has the character A-D within single quotes, which is necessary since it is a char variable that is checked. The default section takes care of all characters other than A, B, C or D.

5.4 Menu Program with Loop

We will now improve our menu program so that the user repeatedly can enter different options without terminating the program. We then put the entire menu printing and switch statement in a loop. The JSP graph will then be:



Click on the ad to read more

Download free eBooks at bookboon.com

We complete the menu with still another option, X – Exit. As long as the user does not enter X, the loop proceeds. Furthermore we also clean the screen before the menu is displayed, which is the first operation of the loop. Here is the code:

```
#include <iostream.h>
#include <stdlib.h>
void main()
{
  char cSel, temp;
  do
  {
    system("cls");
    cout << "   Menu" << endl <<"   ===" << endl;
    cout << "A. Order" << endl;
    cout << "B. Invoice" << endl;
    cout << "C. Warehouse" << endl;
    cout << "D. Finance" << endl;
    cout << "X. Exit" << endl;
    cout << "Select: ";
    cin >> cSel;
    switch (cSel)
    {
      case 'A':
        cout << "You selected Order";
        break;
      case 'B':
        cout << "You selected Invoice";
        break;
      case 'C':
        cout << "You selected Warehouse";
        break;
      case 'D':
        cout << "You selected Finance";
        break;
      case 'X':
        cout << "You selected to exit";
        break;
      default:
        cout << "Erroneous choice";
        break;
    }
    cout << endl << "Press a key to continue";
```


Download free eBooks at bookboon.com

```
    cin >> temp;
  }while (cSel != 'X');
}
```

To be able to clear the screen we need the header file stdlib.h.

W use a do loop, where the condition is checked *after* the loop to ensure that the loop is run at least once.

The first action in the loop is to clear the screen with system("cls"). Then the menu is printed and the user is prompted for an option, i.e. a character to be stored in the variable cSel. That variable is checked in the switch statement, where a text is printed corresponding the selected option. If the user enters 'X', the text 'You selected to exit' will be printed, and the loop is terminated since the while condition specifies that cSel must not equal 'X'.

The method of letting the user enter an extra character to the variable temp at the end of the loop is a relatively unconvenient solution, but it has the advantage of avoiding to struggle with special C++ features regarding input, which we don't go into here.

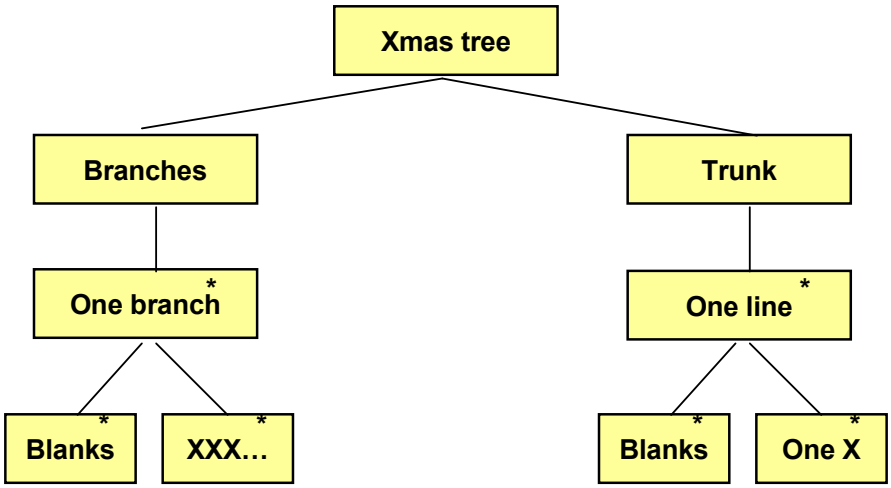
5.5 Christmas Tree

We will now create a logically rather complex program that uses char variables to print a number of 'X' on the screen with the shape of a Christmas tree:

```
            X
           XXX
          XXXXX
         XXXXXXX
        XXXXXXXXX
       XXXXXXXXXXX
      XXXXXXXXXXXXX
     XXXXXXXXXXXXXXX
    XXXXXXXXXXXXXXXXX
   XXXXXXXXXXXXXXXXXXX
  XXXXXXXXXXXXXXXXXXXXX
 XXXXXXXXXXXXXXXXXXXXXXX
            X
            X
```

As you can see the tree has eleven branches and two 'X':s to the trunk. We therefore need an outer loop that runs eleven times, where each turn prints a branch. Each branch consists of a number of 'X':s, different depending on which branch being printed. Furthermore we will have to print a suitable number of blanks before the 'X':s, so that the branches are centered symmetrically around the middle trunk. We therefore need two inner loops, one that prints the leading blanks and one that prints the 'X':s for each branch. After completion of the branches we need a loop that runs two turns and that prints the 'X':s for the trunk. We start with a JSP graph:

Download free eBooks at bookboon.com



The JSP graph tells us that there is one outer loop for the branches, where each turn of the loop prints a branch. We then have one inner loop that prints the correct number of blanks and one inner loop that prints the 'X':s. The same is true for the trunk where we have an outer loop where each turn of the loop prints one line of the trunk, and one inner loop that prints the blanks before one single 'X' is printed. The code is as follows:

Click on the ad to read more
Download free eBooks at bookboon.com

```
#include <iostream.h>
void main()
{
  int i, j;
  char x = 'X', blank = ' ';
  for (i=10; i>=0; i--)  //first outer for-loop
  {
    for (j=0; j<i; j++)  //first inner for-loop
      cout << blank;
    cout << x;
    for (j=0; j<10-i; j++)  //second inner for-loop
      cout << x << x;
    cout << endl;
  }
  for (i=0; i<2; i++)  //second outer for-loop
  {
    for (j=0; j<10; j++)  //inner for-loop
      cout << blank;
    cout << x << endl;
  }
}
```

We declare a variable named x that correspond to the 'X', and a variable named blank corresponding to the blank character.

The first outer for-loop has an i as loop counter and goes from 10 to 0 (11 turns). We have selected to let the values run backwards to make the subsequent math easier. i is consequently a line counter for the branches of the tree.

The first inner loop has a j as loop counter and goes from 0 to i. It prints the correct number of blanks for each branch. Since i is counted reversed, the number of blanks will decrease for each branch. Each turn of the loop prints one blank.

After completion of the blanks for a bransch, first an 'X' is printed. Since the number of 'X':s at each bransch is odd, the remaining number of 'X':s to be printed is even.

The second inner for-loop prints the 'X':s with two 'X':s for each turn of the loop. We again use j as loop counter, which goes from 0 to 10-i. This means that according to the decrease of i, the number of printed 'X':s increases. And since two 'X':s are printed for each turn of the loop, there will always be an even number of 'X':s.

The second outer for-loop, which builds the trunk of the tree, has i as loop counter and goes from 0 to 1 (two turns). For each of the two turns, there is an inner for-loop that goes from 0 to 9 and that prints 10 blanks. After the inner loop, an 'X' is printed.

This complex algorithm probably requires some thinking. You could preferably write a scheme of how i and j changes their values, and in that way follow the progress of the program.

Download free eBooks at bookboon.com

5.6 int - char

Each character has an internal code of the integer type. E.g. the character A has the code 65, B has 66 tec. Therefore the data types integer and char can cooperate. Here is an example:

```
int iNo = 65;  //The code for A
char cChr;
cChr = iNo;
cout << cChr;
```

First we declare an integer variable, iNo, and give it the value 65. On the second line we declare a char bariable named cChr. On the third line cChr gets the same value as iNo, i.e. 65. When we then print cChr on the fourth line, the program understands that it is a char variable being printed, so the character corresponding to the code 65 will be printed, namely A.

5.7 Å, Ä, Ö

As you probably has guessed, the character codes follows the alphabetic order, i.e. A=65, B=66, C=67 etc. The Swedish characters Å, Ä and Ö don't follow that pattern. Therefore you can use the hexadecimal codes for these characters:

```
'\x86'   å
'\x84'   ä
'\x94'   ö
'\x8F'   Å
'\x8E'   Ä
'\x99'   Ö
```

The characters \x indicate that a hexadecimal value will follow. If you for instance want to print the word 'från', write as follows:

```
cout << "fr\x86n";
```

If you want to assign the char variable cChr the value Ö, write this:

```
cChr = '\x99';
```

5.8 String Array, char[]

A limitation with a char variable is that it can only store one single character. Many times you want to store a longer text in a variable like a customer name or a product description. Then you will need a string array. A string array works in the same way as any other array, i.e. it has a variable name and an index value that indicates an item of the array, i.e. a specific character of the string.

Below we declare a string array called cName, which can hold up to 30 characters:

```
char cName[30];
```

The indeces can be any of 0-29.

Suppose we want the user to enter a name to the array cName:

Download free eBooks at bookboon.com

```
cout << "Enter your name: ";
cin >> cName;
```

Let's say that the user enters 'John'. These four characters are then stored in the cName array, where the first item cName[0] contains the character 'J', the second item cName[1] the character 'o', cName[2] the character 'h' and cName[3] the character 'n'.

In addition, an extra character is always stored as the last character of a string. It is the so-called null character '\0', which is stored in the fifth position in the item cName[4].

The program should now print what the user entered:

```
cout << "Your name is " << cName;
```

In our case the text 'Your name is John' will be printed. Note that when an entire string array is printed, you don't need to specify any indeces in the cout statement. The procedure by cout is to print one character after the other in the string array from the first position until the null character is found.

Suppose that the user in the program section above enters his entire name 'John Smith'. When the cout statement is executed, still only the text 'Your name is John' is printed. The reason to this is that when text is entered from the keyboard, only characters up to the first blank will be stored in the array. This is a limitation of the cin statement.

A solution to this problem is to use another input function instead of cin:

```
char cName[30];
cout << "Enter your name: ";
cin.getline(cName,29);
cout << "Your name is " << cName;
```


Download free eBooks at bookboon.com

Here we use the function cin.getline() instead of only cin. This has the effect that, if the user enters 'John Smith', the printout from the cout statement will be 'Your name is John Smith'. Blanks consequently work in the correct way. The input is *not* interrupted at the first blank, but the entire text line entered by the user will be stored in the variable cName.

The function getline() must have two parameters in the parenthesis; the string array to receive the entered text, and the maximum number of characters allowed to receive. The reason for using 29 is to allocate space for the null character at the end of the string as the 30th character. If the user enters less than 30 characters, say 14, the null character will be stored in position 15. If the user enters more than 29 characters, only the first 29 are accepted.

5.9 Length of a String

We will now create a little program that calculates the length of a string, i.e. the number of characters contained in the string. For that purpose we use the function strlen():

```
#include <iostream.h>
#include <string.h>
void main()
{
  char cName[30];
  cout << "Enter your name: ";
  cin.getline(cName,29);
  cout << "Your name is " << cName << endl;
  cout << "Your name is " << strlen(cName) <<
           " characters long" << endl;
  cout << "The entire string is " << sizeof cName <<
           " characters long" << endl;
}
```

To be able to use string functions like strlen() we must include the header file string.h.

In the program we first declare a string array called cName with 29 positions plus the null character, totally 30 characters. Then the user is prompted for his name, which is stored in the variable cName, and a confirmation is printed 'Your name is John Smith'.

In the next last cout statement the text 'Your name is 10 characters long' is printed, provided that we have used a name of 10 characters like 'John Smith'.

In the last cout statement the text 'The entire string is 30 characters long' is printed. Here we use the operator sizeof, which gives the declared length of the string array cName, irrespective of how many characters actually have been stored. Thus, note the difference between strlen() and sizeof.

Download free eBooks at bookboon.com

5.10 Upper/Lower Case

We have previously mentioned that the character codes for the letters A-Z start with 65 and go upwards. This applies to the upper case characters. Lower case letters a-z have the same pattern starting with 97, i.e. 32 greater than the upper case letters. An upper case is consequently achieved by subtracting 32 from the character code of the lower case character.

We will now create a program that reads a text in lower case and converts it to upper case and prints it:

```
#include <iostream.h>
#include <string.h>
void main()
{
  int i;
  char cName[30];
  cout << "Enter your name in lower case: ";
  cin >> cName;
  int iLen = strlen(cName);
  for (i=0; i<iLen; i++)
    cout << (char)(cName[i] - 32);
  cout << endl;
}
```

As usual in string handling we must include the header file string.h.

We declare the variable i to be used as loop counter, and a string array named cName with space for 29 characters. The user is then prompted for his name in lower case, which is stored in the array cName.

We then calculate the length of the input string, which is stored in the variable iLen. This value is used as loop limit in the subsequent loop, which performs as many turns as there are characters in the input string.

Inside the for-loop we take the ith character from the string cName (cName[i]) and decrease it by 32. Since this is a math operation, the program understands that it is the character code for the ith character that is decreased by 32. This value is then type cast to char by placing char within parenthesis in front of the subtraction. Then the program understands that it is the corresponding character that is to be printed.

Thus, we have taken character by character from the input string, all in lower case, decreased the character code by 32, which gives the corresponding upper case character, and printed it.

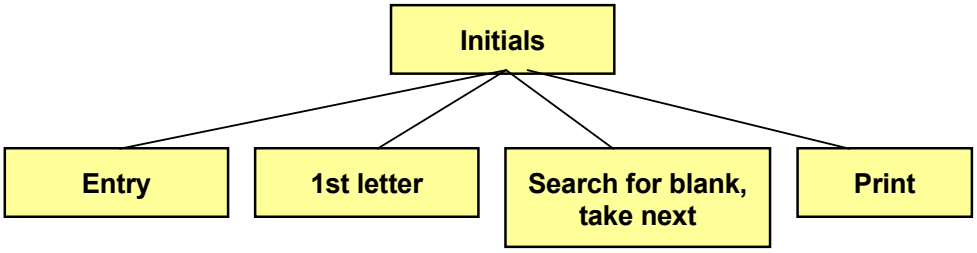
5.11 Initials

In most computer systems the programming work is to a large extent focused on checking and processing texts entered by users. Therefore, we will study some text processing examples..

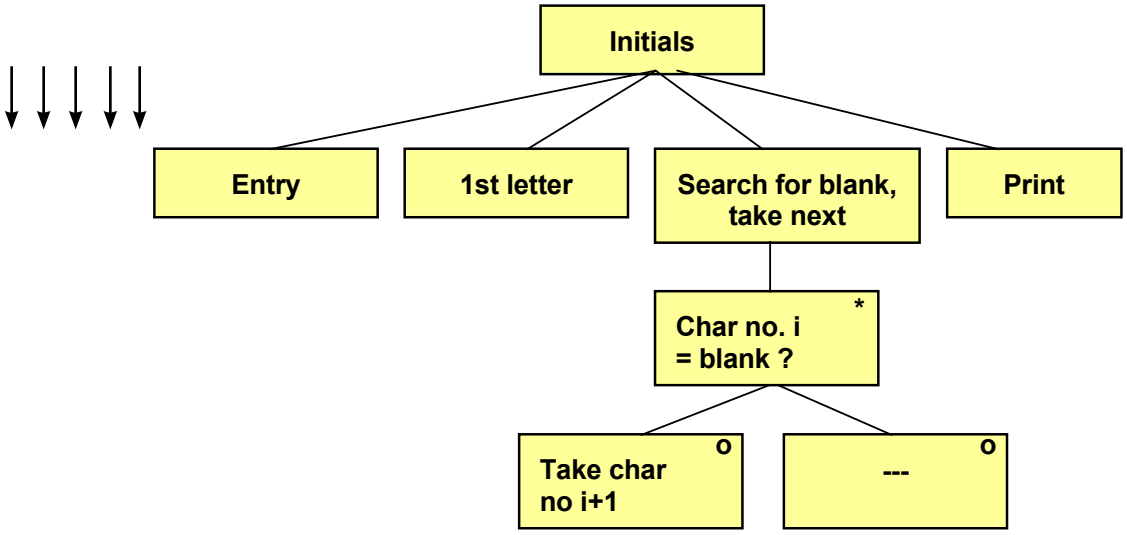
In our next program example we will extract the initials from a name entered by the user. The process is to extract the first letter from the name. Then we search for the blank between the first name and the surname. In the next position the first letter of the surname is found.

Download free eBooks at bookboon.com

We start with a JSP graph:



To search for the blank implies taking one character at a time from the beginning of the input name and check if it is a blank. We detail the 'Search for blank, take next' box:



Searching for the blank is made in a loop. For each turn of the loop we check if character no. i is blank. If so, we take character no. i+1 as the second initial. Here is the code:

```
#include <iostream.h>
#include <string.h>
void main()
{
  int i;
  char cName[30], cInit[3];
  cout << "Enter your name: ";
  cin.getline(cName,29);
  int iLen = strlen(cName);
  cInit[0] = cName[0];
  for (i=1; i<iLen; i++)
    if (cName[i] == ' ')
      cInit[1] = cName[i+1];
```


Download free eBooks at bookboon.com

```
  cInit[2] = '\0';
  cout << "Your initials are " << cInit << endl;
}
```

We declare the string array cName, which is to contain the input name, and cInit to contain the initials. The reason for declaring 3 positions for cInit is to make space for the 2 initials plus the null character.

The user enters his name to the array cName, and its length is calculated and stored in the variable iLen.

The first character of cName (cName[0]) is stored in the first position of cInit.

Then we have the loop with the loop counter i, which represents the index in the input string, and goes from position 2 (index=1) to the end of the string. For each turn of the loop we check if the character is a blank. If so, we copy the next character (cName[i+1]) to the second position of cInit.

At completion of the loop we put the null character in the third position of cInit and print cInit.

5.12 Comparing Two Strings

In the Arrays chapter you learnt that, when comparing two arrays, you must compare each item of the arrays. For strings there is a special function, strcmp(), that can compare two string arrays. The function

```
strcmp(str1, str2)
```

- gives a negative result if str1 < str2, i.e. if str1 comes before str2 in alphabetic order

Click on the ad to read more

Download free eBooks at bookboon.com